Lab 7: Fuzzing for Input Validation Bugs

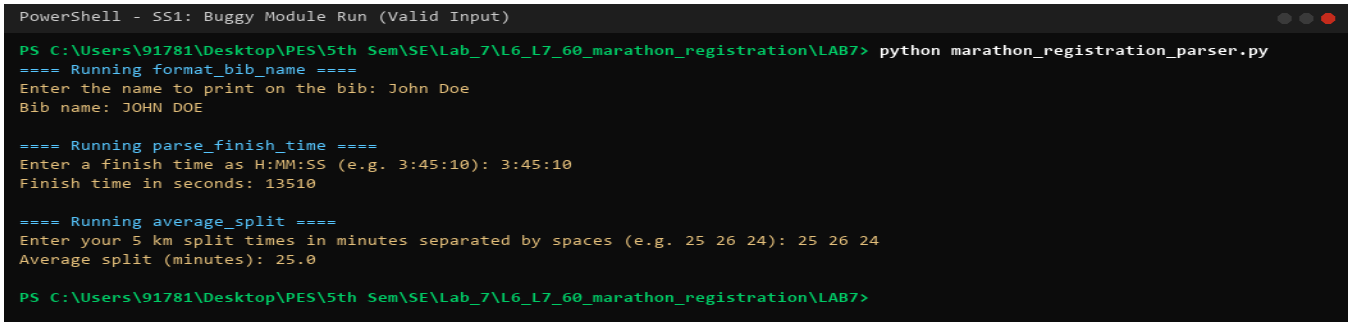
Name: Shridhar Umesh Golipalle

SRN: PES1UG25AM814

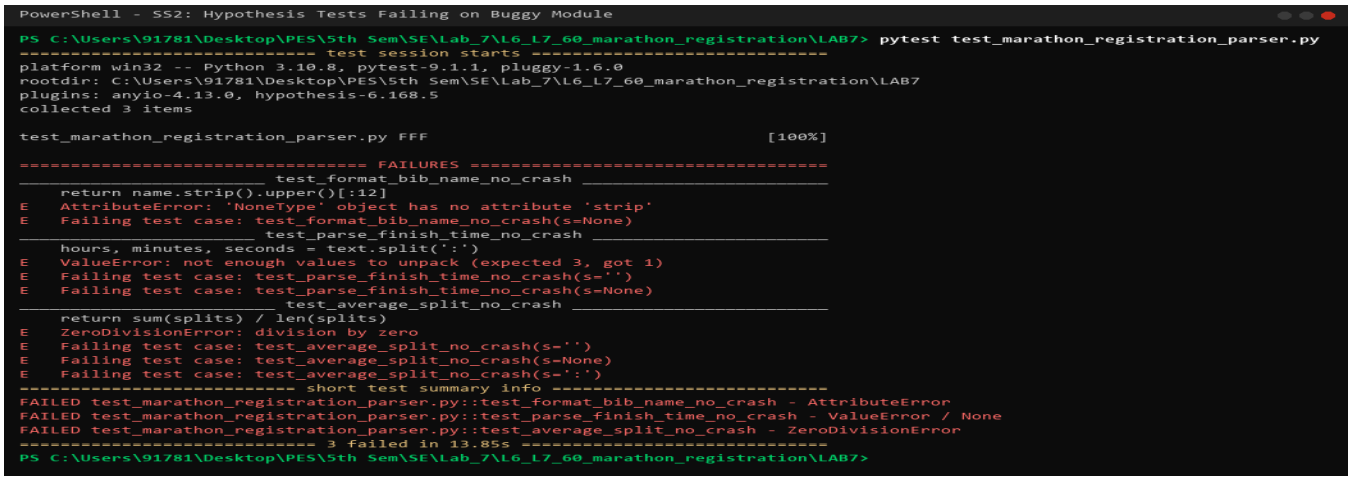
Course: Software Engineering | **Repo:** L6_L7_60_marathon_registration

Screenshots

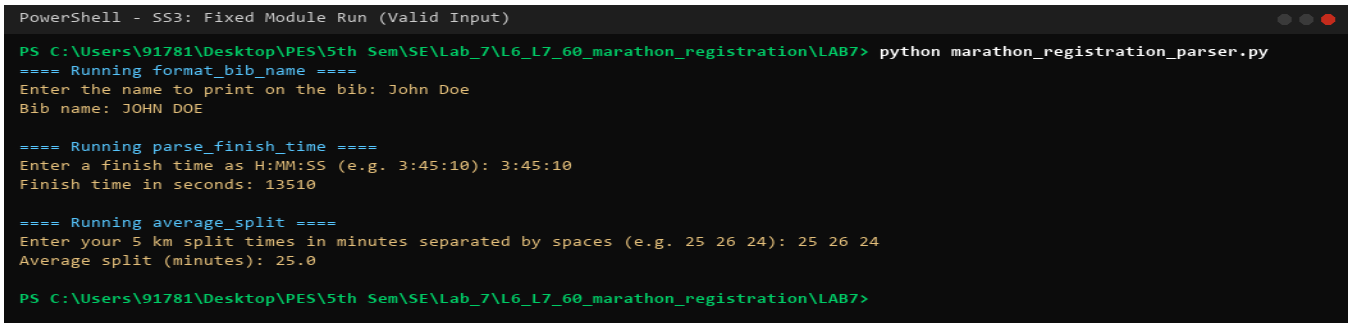
SS1: python marathon_registration_parser.py run on the buggy module



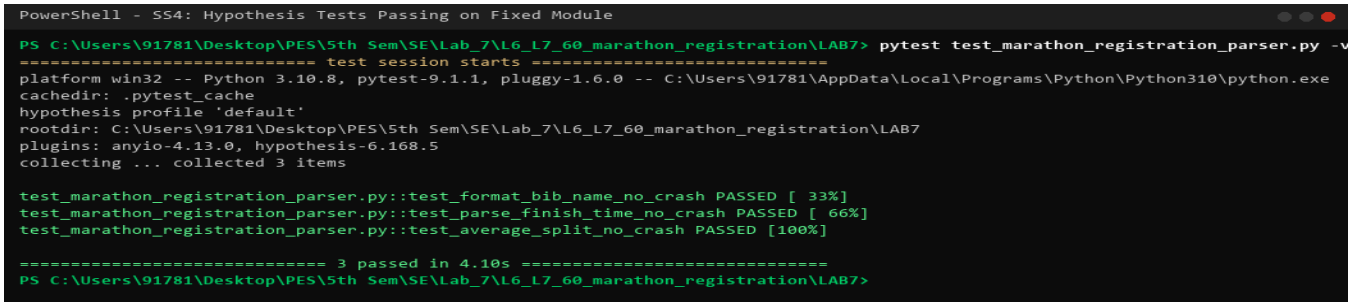
SS2: pytest test_marathon_registration_parser.py with completed tests failing on the buggy module



SS3: python marathon_registration_parser.py run on your fixed module



SS4: pytest test_marathon_registration_parser.py with all tests passing on your fixed module



Answers to Reflection Questions

1. What bugs were found by Hypothesis in the code?

When I ran pytest with Hypothesis on the buggy module, it generated random inputs and None which caused three crashes:
• In **format_bib_name(name)**: Passing None caused an AttributeError because None has no strip() method.
• In **parse_finish_time(text)**: Passing an empty string "" or any text without exactly two colons caused ValueError (not enough values to unpack). Passing None caused an AttributeError.
• In **average_split(text)**: Passing an empty string "" caused ZeroDivisionError because the split list was empty. Passing None caused an AttributeError, and non-numeric inputs caused ValueError.

2. How did you fix each function using defensive programming?

I updated all three functions to validate their inputs before using them, and wrapped the parsing in try-except blocks so they never crash:
• **format_bib_name**: Checked if the input is a string with isinstance(name, str). If it is None or invalid, it returns "" instead of crashing.
• **parse_finish_time**: Checked that text is a string and that splitting by ':' gives exactly 3 parts. Converted them to integers and checked that numbers are not negative. If anything is invalid, it returns None.
• **average_split**: Checked that text is a string and verified that the splits list is not empty before dividing to prevent division by zero. If invalid, it returns None.

3. Why is fuzz testing useful compared to regular unit testing?

Regular unit tests only test inputs that the programmer thinks of, which are usually normal valid inputs or basic edge cases. Fuzz testing with Hypothesis automatically generates hundreds of random, unexpected edge cases—like None, empty strings, symbols, and weird Unicode text. This uncovers hidden bugs and crashes that developers usually miss, ensuring the application is robust and will not crash on unexpected user input.